

COURSE CODE/TITLE:	CPE010	SECTION:	CPE21S1
DATE PERFORMED:	16/7/26	DATE SUBMITTED:	16/7/26
NAME:	Duenas Ralph Florenz F.	INSTRUCTOR:	Engr. Jimlord Quejado

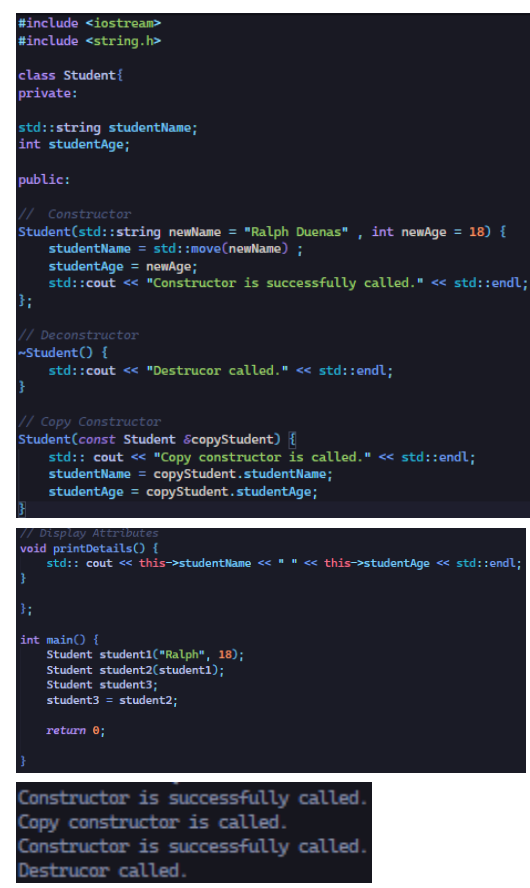
ACTIVITY NO. ____ HOA 2.1

1. PROCEDURE OUTPUT

What procedures did you perform?

- ILOA 2.1

Screenshot



```

#include <iostream>
#include <string.h>

class Student{
private:
    std::string studentName;
    int studentAge;

public:
    // Constructor
    Student(std::string newName = "Ralph Duenas" , int newAge = 18) {
        studentName = std::move(newName) ;
        studentAge = newAge;
        std::cout << "Constructor is successfully called." << std::endl;
    };

    // Destructor
    ~Student() {
        std::cout << "Destructor called." << std::endl;
    }

    // Copy Constructor
    Student(const Student &copyStudent) {
        std::cout << "Copy constructor is called." << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }

    // Display Attributes
    void printDetails() {
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};

int main() {
    Student student1("Ralph", 18);
    Student student2(student1);
    Student student3;
    student3 = student2;

    return 0;
}

```

Constructor is successfully called.
 Copy constructor is called.
 Constructor is successfully called.
 Destructor called.

Observation

- The program first creates student1 using the constructor and gives it the name "Ralph" and the age of 18. Next, student2 is created by copying the values of student1 through the copy constructor. Then, student3 is created using the default constructor, and its values are replaced with the values from student2 using the assignment statement. This shows the difference between creating a copy of an object and assigning values to an existing object. When the program finishes, the destructors are called automatically in reverse order, showing that code cleans up the objects after they are no longer used.

- ILOA 2.2

Screenshot

```
#include <iostream>
#include <string.h>

class Student{
private:
    std::string studentName;
    int studentAge;

public:
    // Constructor
    Student(std::string newName = "Ralph Duenas", int newAge = 18) {
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor is successfully called." << std::endl;
    };

    // Destructor
    ~Student() {
        std::cout << "Destructor called." << std::endl;
    }

    // Copy Constructor
    Student(const Student &c (const char [20])"Copy constructor is called."
        std::cout << "Copy constructor is called." << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }

    // Display Attributes
    void printDetails() {
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};

int main() {
    const size_t j = 5;
    Student studentList[j] = {};
    std::string namesList[j] = {"Dane", "Angela", "Andrew", "Nic", "Jarel"};
    int ageList[j] = {15, 16, 18, 19, 16};

    return 0;
}
```

```
Constructor was called
Constructor was called
Constructor was called
Constructor was called
Constructor was called
Destructor was called
Destructor was called
Destructor was called
Destructor was called
Destructor was called
```

Observation

- In my understanding, the program creates an array that can store five Student objects and also prepares separate arrays for student names and ages. Since no Student objects are actually created using the names and ages, the constructor, copy constructor, and printDetails() function are not used during the program. As a result, the program only creates the arrays and then ends, showing that data has been prepared but has not yet been assigned to the Student objects.

- ILOA 2.3

LOOP A

```
for(int i = 0; i < j; i++){ // loop a
Student *ptr = new Student(namesList[i], ageList[i]);
studentList[i] = *ptr;
}
```

Observation

- In Loop A, the program iterates through each element to dynamically create a new Student object using the new keyword, passing the corresponding name and age from namesList and ageList to the parameterized constructor. The created object is then assigned to the studentList array using the assignment operator. Since the array elements were already initialized beforehand, this process performs an assignment operation rather than invoking the copy constructor.

LOOP B

```
for(int i = 0; i < j; i++){ // loop b
studentList[i].printDetails();
}
```

Observation

- In Loop B, the program loops through the studentList array and calls the printDetails() function for each Student object. This outputs the updated student names and ages stored in each object to the console. It confirms that the values assigned during Loop A were successfully saved into the array.

Screenshot

```
#include <iostream>
#include <string.h>

class Student{
private:

std::string studentName;
int studentAge;

public:

// Constructor
Student(std::string newName = "Ralph Duenas", int newAge = 18) {
studentName = std::move(newName);
studentAge = newAge;
std::cout << "Constructor is successfully called." << std::endl;
};

// Destructor
~Student() {
std::cout << "Destructor called." << std::endl;
}

// Copy Constructor
Student(const Student &copyStudent) {
std::cout << "Copy constructor is called." << std::endl;
studentName = copyStudent.studentName;
studentAge = copyStudent.studentAge;
}

// Display Attributes
void printDetails() {
std::cout << this->studentName << " " << this->studentAge << std::endl;
}
};

int main() {
const size_t j = 5;
Student studentList[j] = {};

std::string namesList[j] = {"Dane", "Angela", "Andrew", "Nic", "Jarel"};
int ageList[j] = {15, 16, 18, 19, 16};

for(int i = 0; i < j; i++){ // loop a
Student *ptr = new Student(namesList[i], ageList[i]);
studentList[i] = *ptr;
}

for(int i = 0; i < j; i++){ // loop b
studentList[i].printDetails();
}

return 0;
}
```

```
Constructor is successfully called.  
Dane 15  
Angela 16  
Andrew 18  
Nic 19  
Jarel 16  
Destructor called.
```

Observations

- Based on my understanding, the program first creates an array of five Student objects using the default constructor, so each object is initially given the default name "Ralph Duenas" and the default age of 18. Next, the program enters the first for loop, where a new Student object is dynamically created for each name and age in the namesList and ageList arrays using the parameterized constructor. Each newly created object is then assigned to the corresponding element in studentList using the assignment operator. Since the objects in studentList already exist, this step performs assignment rather than calling the copy constructor. After all the assignments are complete, the second for loop calls the printDetails() function for every object, displaying the updated names and ages stored in the array. When the program finishes execution, the destructors are automatically called in reverse order for the Student objects, showing that the objects are properly destroyed when they go out of scope.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

```

1  #include <iostream>
2  #include <string>
3
4  class Grocery {
5  private:
6      std::string name;
7      double price;
8      int quantity;
9
10 public:
11
12     Grocery(std::string ralph = "", double prc = 0, int q = 0) {
13         name = ralph;
14         price = prc;
15         quantity = q;
16     }
17
18     Grocery(const Grocery &z) {
19         name = z.name;
20         price = z.price;
21         quantity = z.quantity;
22     }
23
24     Grocery& operator=(const Grocery &g) {
25         if (this != &g) {
26             name = g.name;
27             price = g.price;
28             quantity = g.quantity;
29         }
30         return *this;
31     }
32
33     ~Grocery() {}
34
35     double calculateSum() {
36         return price * quantity;
37     }
38
39     void display() {
40         std::cout << "Name: " << name << std::endl;
41         std::cout << "Total Price: " << price << std::endl;
42         std::cout << "Quantity: " << quantity << std::endl;
43         std::cout << "Total amount: " << calculateSum() << std::endl;
44     }
45 };

```

```

45 };
46
47 class Fruit : public Grocery {
48 public:
49     Fruit(std::string ralph, double prc, int q) : Grocery(ralph, prc, q) {}
50
51     void display() override {
52         std::cout << "[Fruit]" << std::endl;
53         Grocery::display();
54         std::cout << std::endl;
55     }
56 };
57
58 class Vegetable : public Grocery {
59 public:
60     Vegetable(std::string ralph, double prc, int q) : Grocery(ralph, prc, q) {}
61
62     void display() override {
63         std::cout << "Vegetables:" << std::endl;
64         Grocery::display();
65         std::cout << std::endl;
66     }
67 };
68
69 double TotalSum(Grocery* list[], int size) {
70     double total = 0;
71
72     for (int i = 0; i < size; i++) {
73         if (list[i] != nullptr)
74             total += list[i]->calculateSum();
75     }
76
77     return total;
78 }

```

```

80 int main() {
81
82     Grocery* GroceryList[4];
83
84     GroceryList[0] = new Fruit("Watermelon", 20, 17);
85     GroceryList[1] = new Fruit("Lychee", 30, 18);
86     GroceryList[2] = new Vegetable("Sayote", 60, 12);
87     GroceryList[3] = new Vegetable("Carrots", 50, 10);
88
89     std::cout << "Andre's grocery list\n\n";
90
91     for (int i = 0; i < 4; i++) {
92         GroceryList[i]->display();
93     }
94
95     std::cout << "Total amount: " << TotalSum(GroceryList, 4) << std::endl;
96
97     std::cout << "\nDeleting Carrots\n";
98
99     delete GroceryList[3];
100    GroceryList[3] = nullptr;
101
102    std::cout << "\nUpdated Grocery List\n\n";
103
104    for (int i = 0; i < 4; i++) {
105        if (GroceryList[i] != nullptr)
106            GroceryList[i]->display();
107    }
108
109    std::cout << "New Total amount: " << TotalSum(GroceryList, 4) << std::endl;
110
111    for (int i = 0; i < 4; i++) {
112        delete GroceryList[i];
113    }
114
115    return 0;
116 }

```

- OUTPUT

```

Total Price: 30
Quantity: 18
Total amount: 540

Vegetables:
Name: Sayote
Total Price: 60
Quantity: 12
Total amount: 720

New Total amount: 1600

```

- Observations

Based on my understanding, the program first declares an array of four Grocery pointers named GroceryList. It then uses the new operator alongside parameterized constructors to dynamically allocate two Fruit objects ("Watermelon" and "Lychee") and two Vegetable objects ("Sayote" and "Carrots") in heap memory. Next, the program runs a for loop that uses dynamic binding via the overridden virtual void display() function to print each item's specific class header, details, and calculated subtotal, followed by calling the TotalSum() function to compute and output the initial total amount.

Summary of lessons learned

- In this HOA, I learned how constructors set up an object's initial data when it is created, while destructors clean up memory when it is no longer needed. I also learned the difference between using a copy constructor to make a brand-new copy of an object versus using the assignment operator to update an object that already exists. Lastly, I learned how to use pointers, the new and delete keywords for dynamic memory, and how virtual functions let child classes run their own code.

Analysis of the procedure

- In those three activities, the procedure showed how Student objects behave when created, copied, assigned, and destroyed. In ILOA 2.1, the program demonstrated object creation using a parameterized constructor, a copy constructor to duplicate an existing object, and an assignment operator to update an existing object's values. In ILOA 2.2, an array of five Student objects was declared, but because no data was assigned to them, only default constructors and automatic destructors were triggered. Finally, in ILOA 2.3, a for loop dynamically created new Student objects using the new operator, assigned their data to the array using the assignment operator, displayed their details, and let destructors clean up the memory at the end.

Analysis of the supplementary activity

- Based on my understanding, the program first creates an array of four Grocery pointers called GroceryList. It uses the new keyword and constructors to create two Fruit objects ("Watermelon" and "Lychee") and two Vegetable objects ("Sayote" and "Carrots") in heap memory. Next, a for loop runs the overridden display() function for each item using virtual functions to print their details and costs, followed by TotalSum() to calculate the initial total amount.

Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?

- I think I did very well in this HOA. But I think I need more practice to get used to core programming ideas like class inheritance, dynamic memory allocation with pointers, and function overriding with virtual keywords. I was also able to easily follow how constructors, destructors, and assignment operators work step by step.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Python Tutor

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Code assistance, explanation of concept and the flow of the code itself.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

I used Python tutor to elaborate the thoughts I have in my mind without replacing the original thought and also for getting the formulas that are needed. I affirm that my use of AI complies with TIP's AI usage policy.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.